

# Reviewer Pack — Minimal Rank of Kostant Sheaves and $AC^0$ Circuit Lower Bound...

Entry bdbaf85bd98d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 23:53:48 UTC

## Minimal Rank of Kostant Sheaves and $AC^0$ Circuit Lower Bounds

**Entry ID:** bdbaf85bd98d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 23:53:48 UTC

### 1. Conjecture

**Field A** (mathematical branch): Lie Algebras and Representation Theory **Field B** (complexity object):  $AC^0$  Circuit Complexity

#### Statement:

[‘For any given Boolean function  $f$ , the minimal rank of the Kostant sheaf associated with the Lie algebra of its Fourier-Mukai transform is upper bounded by a constant times the size of the smallest  $AC^0$  circuit computing  $f$ .’, ‘Equivalently, for all instances with  $n \leq 40$ , the inequality  $\min_r \text{rank}(\text{KostantSheaf}(\text{Fourier}(f))) \leq c * |\text{CircuitSize}(f)|$  holds, where  $\text{KostantSheaf}$  is computed by applying a constructive mapping from the Boolean function to the representation theory of Lie algebras.’, ‘The conjecture can be falsified by finding an instance where the inequality does not hold for any constant  $c$ .’]

#### Rationale (proposer’s reasoning):

[‘Kostant sheaves provide a natural algebraic object associated with representations of Lie algebras, which may encode combinatorial information about Boolean functions. The rank of these sheaves could potentially reflect the complexity of computing the function in  $AC^0$ .’, ‘Lie algebra representation theory is rarely applied to circuit complexity analysis, and exploring this connection might expose new structures or methods that are difficult to achieve with more traditional approaches.’]

**Taxonomy category:** LieAlgebraRepresentationTheory ×  $AC^0$ CircuitComplexity (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 5a8003b5992d214a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

For  $n \leq 40$  Boolean functions, if the inequality  $\min_r \text{rank}(\text{KostantSheaf}(\text{Fourier}(f))) \leq c * |\text{CircuitSize}(f)|$  holds for all seeds and a fixed constant  $c$ , then the conjecture is supported.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | SAFE             | HITS             |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "Kostant sheaf" AND "AC<sup>0</sup> circuit lower bounds" - "Fourier-Mukai transform" AND "minimal rank Lie algebra" - "Boolean function" AND "Lie algebra representation theory"

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.4s

#### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def circuit_size(f):
        n = len(f)
        if n == 1:
            return 1
        else:
            return 2 + min(circuit_size(f[:n//2]), circuit_size(f[n//2:]))

    def fourier_transform(f):
        n = len(f)
        result = [0] * (2*n)
        for k in range(2*n):
            sum_val = 0
            for j in range(n):
                sum_val += f[j] * math.cos(math.pi * j * k / n) + f[j+n] * math.sin(math.pi * j * k / n)
            result[k] = sum_val
        return result

    def min_rank_kostant_sheaf(f):
        n = len(f)
        fourier_f = fourier_transform(f)
        rank = 0
```

```

    for i in range(n):
        if fourier_f[i] != 0:
            rank += 1
    return rank

n = random.randint(5, 40)
f = generate_boolean_function(n)

rank = min_rank_kostant_sheaf(f)
circuit_size_val = circuit_size(f)

if circuit_size_val == 0:
    return {
        "metric_name": "min_r rank(KostantSheaf(Fourier(f)))",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "Circuit size is zero"
    }

c = 2 # Example constant, can be adjusted
if rank <= c * circuit_size_val:
    return {
        "metric_name": "min_r rank(KostantSheaf(Fourier(f)))",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }
else:
    return {
        "metric_name": "min_r rank(KostantSheaf(Fourier(f)))",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Counterexample: n={n}, rank={rank}, circuit_size={circuit_size_val}"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = next(result["counterexample"] for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

TIMEOUT after 240s

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test timed out before producing data, which means we cannot confirm whether the inequality holds for all seeds and a fixed constant  $c$ . | next: Run the test again with increased time limits to ensure it completes and produces results.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 15619        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 9806         |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8267         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8831         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 44448        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9173         |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10207        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9384         |
| 9 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 29599        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 145333 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in `research/audit/bdbaf85bd98d.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bdbaf85bd98d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/bdbaf85bd98d.tar.gz` (if generated)